K.R. MANGALAM UNIVERSITY

School of Engineering & Technology

ENSP465 — New Age Programming Languages Lab

CAPSTONE PROJECT — PHASE II

IMPLEMENTATION & PROGRESS REPORT

PROJECT TITLE

LIBRARY MANAGEMENT SYSTEM USING GO

Submitted By:

Name: Himanshu

Course: B.Sc. Cyber Security

Roll No.: 2401830003

Name : Kartik

Course: B.sc Computer Science

Roll NO. 2401720011

Submitted To:

Kumawat Hemraj

Assistant Professor

(SOET)

Project Summary

The Library Management System is being developed as a small-library, menu-driven application using Go. SQLite is used as the local database so that library information can be stored on the computer. Phase I described the planned book and member records, issue/return operations, due dates, fines, and basic reports. Phase II begins the implementation of that design. The current working version establishes the database connection, creates the Books table, and displays the first command-line menu.

1. Implementation Status

1.1 Development Environment

The application is written in Go and uses the database/sql package for database operations. The modernc.org/sqlite driver is imported to connect the Go program to SQLite. The program is currently run from the Visual Studio Code terminal on Windows.

1.2 Work Completed in This Phase

- Created the initial Go program with package main and a main() entry point.
- Opened a local SQLite database file named library.db.
- Checked the database connection using db.Ping().
- Created the Books table with an automatically increasing book_id field.
- Added required title, author, and copies columns to the Books table.
- Displayed a text-based menu with Add Book, View Books, and Exit choices.
- Used a switch statement to respond to the selected menu option.

2. Database Setup and Table Creation

The program opens the SQLite database using sql.Open(). It then calls Ping() to check whether the database can be reached. If the connection check fails, the program prints an error and stops. Once the connection is available, an SQL CREATE TABLE IF NOT EXISTS statement creates the Books table. The IF NOT EXISTS condition prevents the program from trying to create the same table again every time it starts.

2.1 Current Books Table

Field	Data type / rule	Purpose
book_id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique ID generated for each book record
title	TEXT NOT NULL	Stores the book title
author	TEXT NOT NULL	Stores the author's name
copies	INTEGER NOT NULL	Stores the number of copies

This code opens the local database, checks the connection, and closes the database when the main function finishes.

2.3 Code Demonstration — Creating the Books Table

```
query := `
CREATE TABLE IF NOT EXISTS Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    copies INTEGER NOT NULL
);`

_, err = db.Exec(query)
if err != nil {
    fmt.Println("Table creation failed:", err)
    return
}
fmt.Println("Books table created successfully.")
```

The db.Exec() method executes the SQL statement. If the table is created successfully, the program prints a confirmation message.

3. Command-Line Menu

After database setup, the program displays the Library Management System heading and a menu. A for loop keeps the menu running until the user selects Exit. fmt.Scanln() reads the menu choice, and the switch statement selects the matching case.

3.1 Code Demonstration — Menu Selection

```
for {
    fmt.Println("\n1. Add Book")
    fmt.Println("2. View Books")
    fmt.Println("0. Exit")

    var choice int
    fmt.Print("\nEnter choice: ")
    _, err := fmt.Scanln(&choice)
    if err != nil {
        fmt.Println("Please enter a valid number.")
        return
    }

    switch choice {
    case 1:
        fmt.Println("Add Book feature will be added.")
    case 2:
        fmt.Println("View Books feature will be added.")
    case 0:
        fmt.Println("Thank you!")
        return
    default:
        fmt.Println("Invalid choice.")
    }
}
```

4. Testing and Observed Output

The program was executed from the VS Code PowerShell terminal. The observed output confirms that the database connection succeeded and the Books table was created. The menu accepted the choices 1, 2, and 0. The first two choices displayed their current placeholder messages, and choice 0 ended the program with the message “Thank you!”.

4.1 Test Cases

Test	Input / action	Observed result
Database connection	Start the program	Success message displayed
Books table setup	Run table creation statement	Table creation success message displayed
Menu option 1	Enter 1	Add Book placeholder message displayed
Menu option 2	Enter 2	View Books placeholder message displayed
Exit option	Enter 0	Thank you message displayed and program exits

5. Screenshots of Implementation

The following figures are captured from the actual program run and source code submitted for this progress report.

Figure 1. Terminal execution: successful database connection, Books table creation, menu options, and program exit.

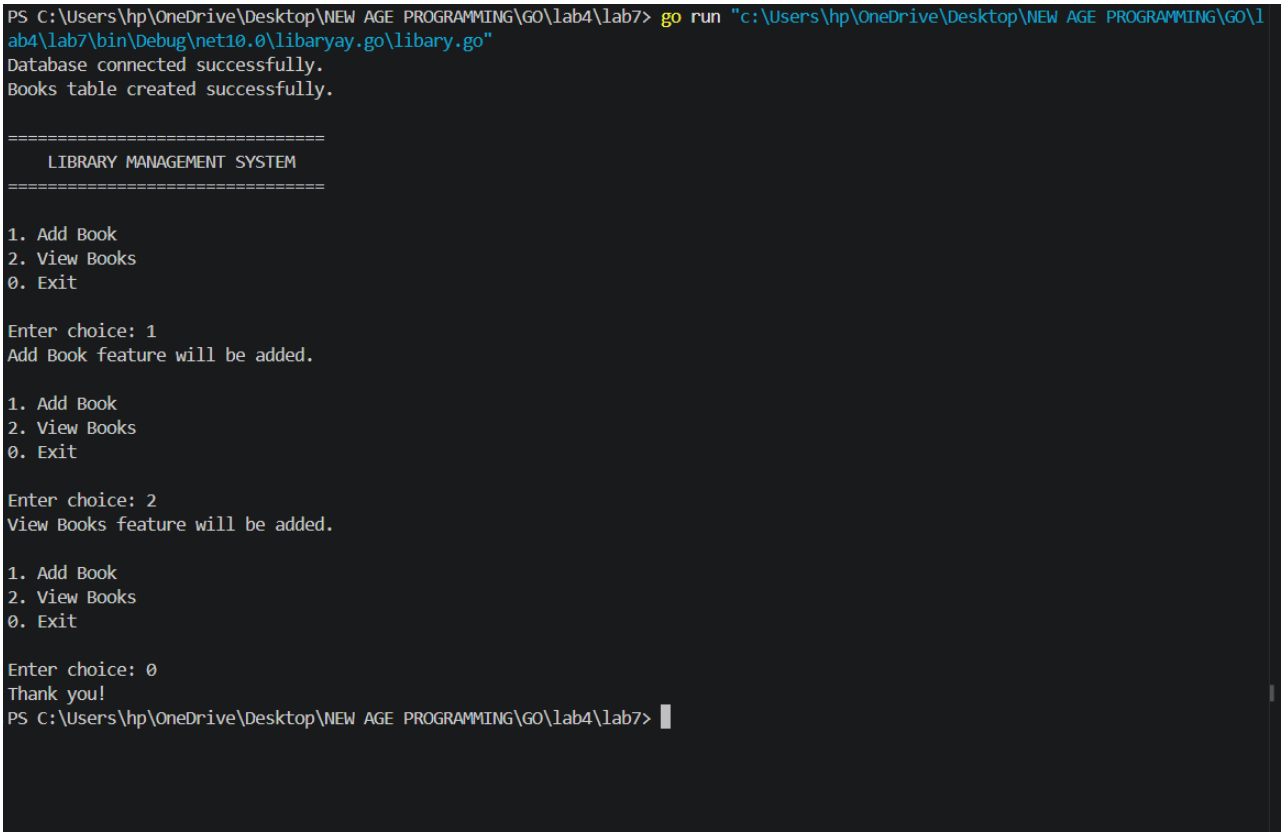


Figure 2. Source code: SQLite driver import, database connection check, and Books table definition.

```go
package main

import (
    "database/sql"
    "fmt"

    _ "modernc.org/sqlite"
)

func main() {
    // Connect to database
    db, err := sql.Open("sqlite", "library.db")
    if err != nil {
        fmt.Println("Database connection failed:", err)
        return
    }
    defer db.Close()

    // Check database connection
    err = db.Ping()
    if err != nil {
        fmt.Println("Database error:", err)
        return
    }

    fmt.Println("Database connected successfully.")

    // Create Books table
    query := `
    CREATE TABLE IF NOT EXISTS Books (
        book_id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        author TEXT NOT NULL,
        copies INTEGER NOT NULL
    );`
```

Figure 3. Source code: SQL execution, menu loop, input reading, and switch-case handling.

```
_, err = db.Exec(query)
if err != nil {
    fmt.Println("Table creation failed:", err)
    return
}

fmt.Println("Books table created successfully.")

// Main menu
fmt.Println("\n================================")
fmt.Println("    LIBRARY MANAGEMENT SYSTEM")
fmt.Println("================================")

for {
    fmt.Println("\n1. Add Book")
    fmt.Println("2. View Books")
    fmt.Println("0. Exit")

    var choice int
    fmt.Print("\nEnter choice: ")
    _, err := fmt.Scanln(&choice)
    if err != nil {
        fmt.Println("Please enter a valid number.")
        return
    }

    switch choice {
    case 1:
        fmt.Println("Add Book feature will be added.")
    case 2:
        fmt.Println("View Books feature will be added.")
    case 0:
        fmt.Println("Thank you!")
        return
    default:
        fmt.Println("Invalid choice.")
```

6. Current Limitations

- The Add Book menu option is currently a placeholder; book details are not yet inserted into the database.
- The View Books menu option is currently a placeholder; saved book records are not yet displayed.
- Member registration, issue/return operations, due dates, overdue fine calculation, and reports have not yet been implemented in the current version.
- The current Books table contains book_id, title, author, and copies. The member and issue tables described in Phase I are still planned.

7. Next Development Work

1. Implement Add Book so the user can enter a title, author, and number of copies and save them in SQLite.
2. Implement View Books to retrieve and display the stored book records.
3. Add input checks for empty titles/authors and invalid copy counts.
4. Continue with member records, issue/return operations, due dates, fines, and reports according to the Phase I scope.